

CMPU3010 Databases 2 — Week 2 Lab

SQL Refresher and Introduction to Advanced SQL – Solution

Part A — Getting Started

A1

patient_id connects an appointment to a patient. It is a foreign key which provides a way of connecting data in the appointment table to data in the patient table.

A2

```
SELECT
    appointment_id,
    patient_id,
    appointment_date,
    status
FROM appointment
WHERE status = 'Completed'
ORDER BY appointment_date;
```

A3 — Optional

```
SELECT
    appointment_id,
    patient_id,
    TO_CHAR(appointment_date, 'DD-Mon-YYYY') AS appointment_date,
    status
FROM appointment
WHERE status = 'Completed'
ORDER BY appointment_date;
```

Part B — Joining Tables

B1

```
SELECT
    a.appointment_id,
    p.first_name,
    p.last_name,
    a.appointment_date,
    a.status
FROM appointment a
JOIN patient p
    ON a.patient_id = p.patient_id
ORDER BY a.appointment_date;
```

B2/B3

```
SELECT
    p.first_name,
    p.last_name,
    s.service_name,
    a.appointment_date,
    a.status
FROM appointment a
JOIN patient p
```

```

        ON a.patient_id = p.patient_id
JOIN service s
        ON a.service_id = s.service_id
WHERE a.status = 'Completed'
ORDER BY a.appointment_date;

```

B4 — Patients with no appointments

```

SELECT
    p.patient_id,
    p.first_name,
    p.last_name,
    a.appointment_id
FROM patient p
LEFT JOIN appointment a
    ON p.patient_id = a.patient_id
WHERE a.appointment_id IS NULL
ORDER BY p.last_name, p.first_name;

```

LEFT JOIN keeps all patients. Patients without a matching appointment have NULL for the appointment columns.

1. Are patients with no appointments included?

Yes. A LEFT JOIN keeps all patients, even when there is no matching appointment.

2. What appears in appointment_id for these patients?

NULL.

3. What happens if LEFT JOIN is changed to JOIN?

Patients without appointments disappear because an INNER JOIN only returns matching rows.

To display only patients with not appointments:

```

SELECT
    p.patient_id,
    p.first_name,
    p.last_name,
    a.appointment_id
FROM patient p
LEFT JOIN appointment a
    ON p.patient_id = a.patient_id
WHERE a.appointment_id IS NULL
ORDER BY p.last_name, p.first_name;

```

B5 — Optional: Services with no bookings

```

SELECT
    s.service_id,
    s.service_name,
    a.appointment_id
FROM appointment a
RIGHT JOIN service s
    ON a.service_id = s.service_id
WHERE a.appointment_id IS NULL
ORDER BY s.service_name;

```

RIGHT JOIN keeps all services because service is the table on the right.

1. Are services that have never been booked included?

Yes.

2. What appears in appointment_id?

NULL.

3. Why does RIGHT JOIN include all services?

A RIGHT JOIN keeps every row from the table on the right, even when there is no matching row on the left.

4. Which is the right table?

service.

To display only services with no bookings:

```
SELECT
    s.service_id,
    s.service_name,
    a.appointment_id
FROM appointment a
RIGHT JOIN service s
    ON a.service_id = s.service_id
WHERE a.appointment_id IS NULL
ORDER BY s.service_name;
```

Part C — Aggregating Data

C1

GROUP BY service_id creates one group for each service so that its appointments can be counted.

C2

```
SELECT
    s.service_name,
    COUNT(*) AS number_of_appointments
FROM appointment a
JOIN service s
    ON a.service_id = s.service_id
GROUP BY s.service_id, s.service_name
ORDER BY number_of_appointments DESC;
```

C3

```
SELECT
    s.service_name,
    COUNT(*) AS number_of_appointments
FROM appointment a
JOIN service s
    ON a.service_id = s.service_id
GROUP BY s.service_id, s.service_name
HAVING COUNT(*) > 5
ORDER BY number_of_appointments DESC;
WHERE filters individual rows; HAVING filters the groups produced by GROUP BY.
```

Part D — Subquery

```
SELECT
    service_name,
    standard_fee
FROM service
```

```
WHERE standard_fee > (
    SELECT AVG(standard_fee)
    FROM service
)
ORDER BY standard_fee DESC;
```

1. What does the inner query return?

One value: the average standard_fee of all services.

2. How is this value used by the outer query?

Each service's standard_fee is compared with the average. Only services costing more than the average are returned.

3. Does the inner query need information from the outer query?

No. It can run independently

Part E — Correlated Subquery

E1.

Which aggregate function finds the latest earlier date? MAX().

For example: MAX(appointment_date) returns the latest date from the matching earlier appointments.

E2.

Correlated subquery:

```
SELECT
    a.patient_id,
    a.appointment_date,
    (
        SELECT MAX(previous.appointment_date)
        FROM appointment previous
        WHERE previous.patient_id = a.patient_id
          AND previous.appointment_date < a.appointment_date
    ) AS previous_appointment
FROM appointment a
ORDER BY
    a.patient_id,
    a.appointment_date;
```

1. Why must previous.patient_id = a.patient_id?

We want the previous appointment for the **same patient**, not an appointment belonging to somebody else.

2. Why use previous.appointment_date < a.appointment_date?

Only appointments occurring before the current appointment can be considered previous appointments.

3. Why use MAX(previous.appointment_date)?

There may be several earlier appointments. MAX() finds the latest of those dates — the appointment immediately preceding the current one.

4. Why do some rows contain NULL?

There is no earlier matching appointment for that patient, so MAX() has no matching dates and returns NULL.

Part F — Window Functions

F1

ROW_NUMBER() numbers each patient's appointments in date order.

- PARTITION BY patient_id starts the numbering again for each patient.
- ORDER BY appointment_date determines the order.
- Without PARTITION BY, all appointments are numbered as one group.

F2

LAG(appointment_date) retrieves the previous appointment date.

- PARTITION BY patient_id ensures it only looks within the same patient.
- ORDER BY appointment_date determines what "previous" means.
- The first appointment for each patient has NULL because there is no previous row.

F3

RANK() ranks services according to their appointment count:

```
RANK() OVER (  
    ORDER BY COUNT(a.appointment_id) DESC  
)
```

If two services have the same number of appointments, they receive the **same rank**.

ROW_NUMBER() gives every row a unique number. RANK() gives tied rows the same rank, so the next rank is skipped.